Design and Formal Verification of a Load–Store Unit with Integrated MESI Cache Coherence FSM

Aayush Patidar
Roll No.: B23479

Abstract

This report presents the design, implementation, simulation, and formal verification of a Load–Store Unit (LSU) integrated with a simplified MESI-based cache coherence protocol. The design is modeled in Verilog and tested using Xilinx Vivado. In addition, a formal verification model is constructed in NuSMV, modeling MESI transitions as a directed graph and expressing cache coherence properties in Temporal Logic (CTL). Symbolic reachability analysis is performed, and counterexamples are interpreted. This project demonstrates how classical hardware protocols such as MESI can be validated using both RTL simulation and model checking.

Contents

1 Introduction **2**

2 MESI Protocol Overview **2**

3 Integrated LSU + MESI Architectural Overview **3**

4 RTL Implementation **3**

5 Simulation and Results (Vivado) **4**
5.1 Console Output 4
5.2 Waveform Screenshot 4

6 Formal Verification Using NuSMV **5**
6.1 Cache State Directed Graph 5
6.2 CTL Properties 5
6.3 NuSMV Execution and Counterexamples 6

7 Conclusion **6**

1 Introduction

Modern multi-core processors require coherent caches to ensure that memory accesses by different cores observe consistent values. One widely used protocol is MESI (Modified, Exclusive, Shared, Invalid). The goal of this project is to design a simple Load–Store Unit (LSU) that interacts with a direct-mapped data cache using a MESI per-line state machine, and then verify correctness using both simulation and model checking.

Two major components are addressed:

- **RTL-level implementation:** A blocking LSU with a single-entry MSHR and a MESI-managed cache. The design supports LOAD, STORE, BusRd, BusRdX, BusUpgr, and snoops.
- **Formal verification:** A symbolic SMV model of a 2-cache MESI system is created. Cache line states are modeled as a directed graph. CTL properties are written and model-checked.

The RTL implementation and the formal model are kept modular and simple, with the focus on correctness rather than performance.

2 MESI Protocol Overview

The MESI protocol marks each cache line with one of four attributes [1]:

- **Modified (M):** The cache holds the only updated copy. Memory is stale.
- **Exclusive (E):** The cache holds the only clean copy, coherent with memory.
- **Shared (S):** Multiple caches may hold the line, all clean.
- **Invalid (I):** Cache line is not valid.

The MESI state machine enforces transitions such as:

- Load miss: `I → E` (via BusRd)
- Store miss: `I → M` (via BusRdX)
- Store hit in Shared: `S → M` (via BusUpgr)
- Snooped BusRdX: `M/S/E → I`
- Snooped BusRd: `M/E → S`

As shown in Figure 1, the MESI protocol uses four cache-line states and well-defined transitions driven by CPU requests and snoop bus events. Each transition ensures coherence is maintained across all cores.

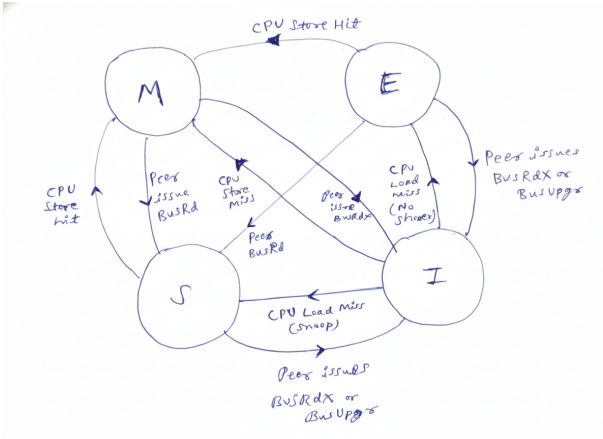

Figure 1: Finite state machine (FSM) of the MESI cache-coherence protocol illustrating all transitions among the Invalid (I), Shared (S), Exclusive (E), and Modified (M) states based on CPU events and snoop responses.

3 Integrated LSU + MESI Architectural Overview

The overall architecture integrates:

1. **Load–Store Unit FSM** Handles CPU requests: `IDLE → CHECK → HIT/MISS → ISSUE → WAIT → FILL → RESPOND`
2. **Cache line storage**
   - Direct-mapped
   - Per-line MESI bits
   - Per-line tag + valid bit
   - Flattened data array
3. **Bus interface** Supports:
   - BusRd
   - BusRdX
   - BusUpgr
   - Bus response with line data
4. **Snoop interface** Receives `snoop_busrd`, `snoop_busrdx`, and `snoop_addr`.

4 RTL Implementation

The RTL implementation includes:

- Blocking LSU with next-state logic
- Tag compare, set selection, and MESI transition logic
- Immediate store-hit response for `E` and `M`

- Miss handling using BusRd or BusRdX
- Upgrade mechanism for `S → M`
- Full snoop handling

5 Simulation and Results (Vivado)

A Verilog testbench is written to test:

1. **Load Miss → BusRd → Fill to E**
2. **Store Hit in E → M**

Below are placeholders for waveform captures and console output.

5.1 Console Output

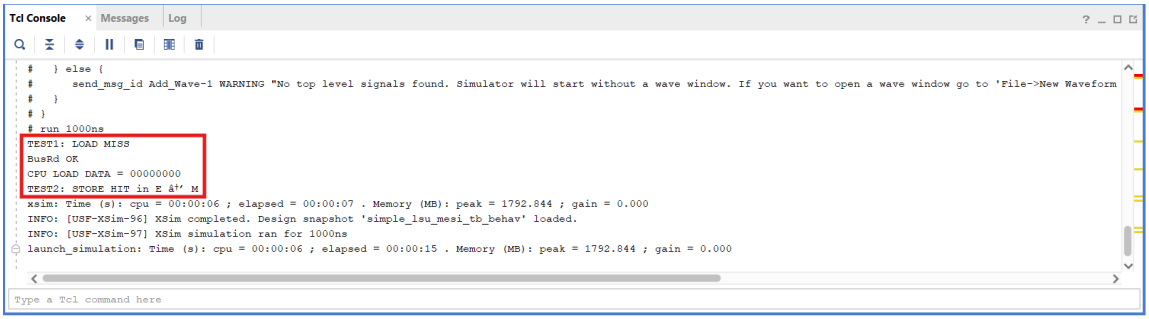

Figure 2: Vivado TCL console output.

5.2 Waveform Screenshot

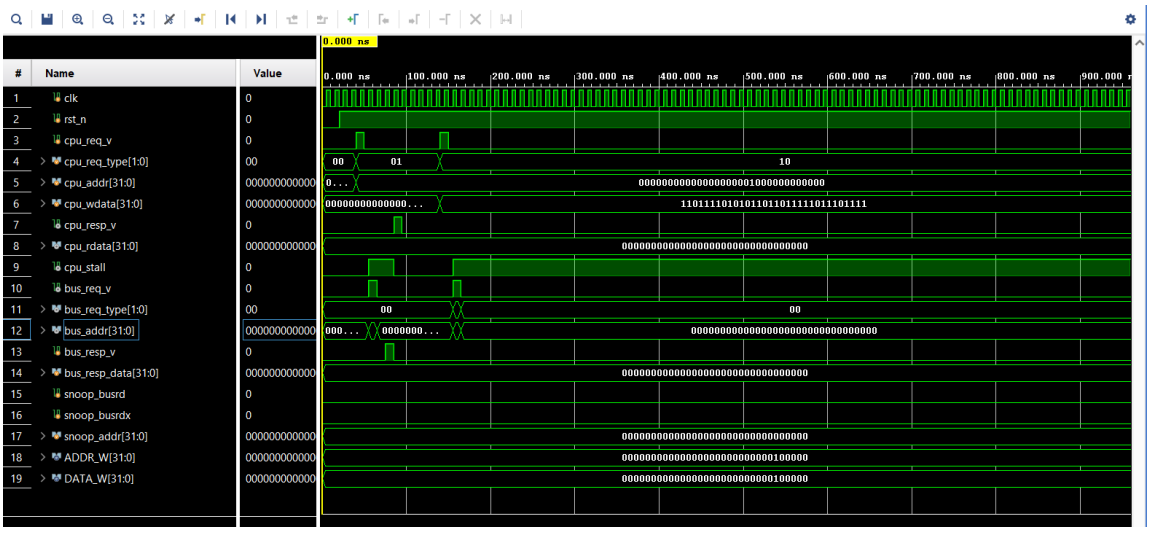

Figure 3: Vivado simulation waveform.

6 Formal Verification Using NuSMV

To formally verify coherence properties, a 2-cache MESI system is modeled in NuSMV. Important aspects:

- Single memory address modeled
- Two caches with MESI state machines
- Simplified bus arbitration
- Symbolic reachability analysis
- CTL properties for correctness

6.1 Cache State Directed Graph

The MESI states of each cache line form a directed graph where edges represent transitions under events such as:

- CPU load/store
- BusRd, BusRdX
- Snoop invalidations
- Fills and writebacks

Symbolic reachability allows NuSMV to generate all possible state combinations of two caches (up to 16 combinations of MESI pairs).

6.2 CTL Properties

The following CTL properties are included:

1. **Correct hit/miss behavior**

$$AG\big(\text{load hit} \rightarrow AX(\text{correct response})\big) \tag{1}$$

2. **Eventual response for misses**

$$AG(\text{load miss} \rightarrow AF(\text{response}))$$

3. **Proper replacement / writeback**

$$AG(\text{evict M} \rightarrow AF(\text{writeback}))$$

4. **Data consistency**

$$AG(\text{response} \rightarrow \text{value} = \text{last write})$$

5. **Uniqueness of Modified**

$$AG\neg(mesi_0 = M \wedge mesi_1 = M)$$

6.3 NuSMV Execution and Counterexamples

When executing:

```
nusmv two_cache_model.smv
```

Several properties fail. Counterexamples reveal:

- CPU requests arrive nondeterministically and simultaneously.
- Bus arbitration is abstract and allows unrealistic interleavings.
- MESI transitions in the abstract model allow illegal "simultaneous M" outcomes.

These results demonstrate:

- The model checker explores every possible sequence, including races that the RTL does not allow.
- Counterexamples are valuable in identifying missing constraints or abstractions in the formal model.

This is expected for an intentionally simplified formal model.

7 Conclusion

This project demonstrates the end-to-end creation and verification of a Load–Store Unit tightly coupled with a MESI protocol engine. The RTL design is functionally validated through simulation. The formal NuSMV model provides symbolic reachability analysis and CTL-based verification of coherence properties.

The NuSMV counterexamples show that care must be taken when abstracting bus behavior and concurrency; otherwise, the state space may admit transitions that real hardware prevents. Nevertheless, this exercise offers an excellent foundation in combining hardware implementation with formal verification techniques.

References

References

[1] ARM Developer Documentation, "MESI and MOESI protocols", https://developer.arm.com/documentation/den0013/0400/Multi-core-processors/Cache-coherency/MESI-and-MOESI-protocols.

[2] NuSMV Model Checker Documentation, https://nusmv.fbk.eu